

Distributed computing attacks on cryptographic systems

A P L Selkirk and A E Escott

It is now possible to link large numbers of desktop computers to achieve massive computing power. This paper briefly describes approaches taken to attack cryptographic algorithms using this method. The approach taken by BT Laboratories in the successful attack on an elliptic curve cipher is detailed. Finally a proposal for the architecture needed to recruit unused computing power for similar problems is given.

1. Introduction

Cryptography is a cornerstone of electronic security and a thriving research area. Attacks on algorithms can take advantage of the increasing power of computers which, according to Moore's law, doubles every eighteen months. Until recently the processing power necessary to break cryptographic codes was achieved by building customised hardware or by using expensive supercomputers. Today computers are powerful enough to use only a fraction of their processing power for everyday purposes. Modern networking technology allows many computers to link together to solve large distributive problems. The computing power that can be achieved by an Internet-wide scheme is potentially astounding. As more computers join the Internet, and more advanced computers are constructed, this power is continually increasing. Therefore it is essential that designers and users of cryptographic algorithms understand what is feasible for a distributed attack to achieve, both now and in the years to come. The Security Research group at BT Laboratories (BTL) has played a major role in recent distributed computing attacks on cryptographic systems, gaining important knowledge both in cryptography and distributed computer processing. This paper outlines what has been achieved in both areas.

2. Attacks on cryptosystems

A cryptographic algorithm is a mathematical method of turning a message into an unreadable series of bytes and converting it back to the actual message again. Because the details of an algorithm can be reverse-engineered, the security of a cryptosystem lies in the secret key that the users share. Changing the key produces different encrypted messages. There are two basic types of cryptographic system, symmetric and public key. In a symmetric key cryptosystem, the same key is used to both encrypt and decrypt the data, so it is vital that it remains secret. In a public key cryptosystem, there are two keys, one to encrypt data and the other to decrypt. The decryption key must remain secret, but

the encryption key can be made known, so that anyone can send encrypted data, but only one person can decrypt it. Digital signatures also use public key cryptography. Here one person signs some data by encrypting it with his secret key, while everyone else can verify the signature using the corresponding public key. For an introduction to cryptography, see Stinson [1].

In cryptography it is important to understand how much computing power is required to break a cryptosystem. The obvious method to attack a cryptosystem is an exhaustive search of the possible keys. If a message and its corresponding ciphertext are known, every key can be used to encrypt the message until the matching ciphertext is found.

The time needed for this attack depends on the number of possible keys, 2^n for a length n binary key and the number of encryptions per second r . This gives a total time to exhaust keyspace of $2^n/r$ seconds with an expected time to find the key of $2^{n-1}/r$. The keyspace can easily be divided so that computers can work in parallel trying different keys. This reduces the time by a factor of the number of computers m with r as the average number of encryptions per second over all the machines.

There have been several large-scale key-cracking efforts on symmetric key cryptosystems over the last couple of years. In 1997, BTL participated in Distributed Computing Technologies' [2] attack on RC5-56. As the name suggests, this is a cipher with a key size of 56 bits, giving 2^{56} or around 72 000 000 000 000 000 keys. The key was found on 19 October after over 200 days. The highest rate achieved was 5300 000 000 keys a second (5.3 Gk/s). The rate for its current project (RC5-64) is now over 50 000 000 000 keys a second (50 Gk/s). As a Pentium-II 300 processor can break around a million keys a second, the processing power is equivalent to about 50 000 such processors.

Recently two challenges involving the 56-bit DES algorithm [3] have been completed. The first was by Distributed Computing Technologies [2], again using computers across the Internet, and took roughly 40 days. The more recent one was completed by the Electronic Frontier Foundation [4] within 3 days using purpose-built hardware which cost less than \$250 000. As DES was designed in the 1970s for the fast encryption of data in hardware, a hardware attack on it is easier¹.

The attacks on public-key cryptosystems are much faster (for the same key length) than attacks on symmetric key and therefore the keys in public-key cryptosystems are longer. This is because there is a mathematical relationship between the public and secret keys. For example, the RSA algorithm has the following relationship between encryption key e and decryption key d :

$$ed = 1 \text{ modulo } (p-1)(q-1)$$

where $n = pq$ is known but its prime factors p and q are secret. If n is factored, it is easy to calculate $(p-1)(q-1)$ and hence the private key. However, factoring does not require n steps, since the primitive factorisation method of checking each odd number between 1 and $\sqrt{n}$ only takes $O(\sqrt{n})$ steps. The best factoring algorithms take approximately $\exp(O((\log n)^{1/3} (\log \log n)^{2/3}))$ steps. In April 1996 an RSA challenge [3] was completed when a 130 decimal digit number (about 430 bits) was factorised. The total computing effort was roughly 500 MIPS years. Common RSA key sizes are 512, 768 or 1024 bits.

An elliptic curve cryptosystem is another public-key cryptosystem. Its attraction is that it has a smaller key-length than RSA for the same level of security, as the best known attacks are slower. An elliptic curve cryptosystem of roughly 170 bits is equivalent to a 1024-bit RSA key. This reduces the amount of storage and computation required for signing, which is important for use in smart cards. In November 1997, Certicom [5] released a series of elliptic curve challenges. The largest one completed so far is the ECCp-97 challenge, which has a 97-bit key, and the attempt was led by a group from BTL [6]. The following section gives more details of elliptic curve algorithms and of this distributive computing effort.

3. The ECCp-97 challenge

Both Miller [7] and Koblitz [8] suggested the use of elliptic curves for cryptography. Given an elliptic curve, there is an operation that allows two points on that curve to be added together to form a third point. This operation enables a cryptosystem to work over some subset of the points on an elliptic curve. To provide the best security the

curve and subset of points are chosen so that the number of points is a large prime n . The security of an elliptic curve cryptosystem relies on the difficulty of finding d where $Q = dP$ and P and Q are points on the elliptic curve (this is known as the discrete logarithm problem). The fastest and most practical method for doing this over a general elliptic curve is the 'parallel Pollard rho method' suggested by van Oorschot and Wiener [9]. The basic concept of the method is to find $X = a_iP + b_iQ$ and $X = a_jP + b_jQ$ for some point X on the curve. Once these equations have been solved, it is easy to solve for d (in virtually every case). To find these equations a set D of distinguished points on the elliptic curve are chosen. These points have no special properties other than it is easy to check whether a point is distinguished or not. Each client randomly generates a_0 and b_0 and calculates the point $X_0 = a_0P + b_0Q$. From this point a sequence of $X_i = a_iP + b_iQ$ are generated using an iteration function (fixed over all clients) until the client finds some X_i in D . The X_i and associated a_i and b_i are passed back to the server, which either calculates the answer if it has seen the point before or stores the triple (X_i, a_i, b_i) in a database of distinguished points. If n is prime the expected running time of the parallel Pollard rho method is approximately $\sqrt{\pi \cdot n/2}$ elliptic curve additions.

The parallel Pollard rho algorithm has several characteristics which make it a very natural problem to distribute over many machines. Firstly, each client repeats a small loop of code and requires virtually no data to run. Secondly, each client runs independently of the rest of the system and the only communication needed is to pass data from the clients to the server. Finally, the server can use the submitted data to reconstruct the point and check it is valid. This prevents invalid results being stored by the server.

During the ECCp-97 challenge, clients inside the BT firewall generally communicated directly with the server using a TCP/IP connection. If the server was down, some clients communicated with a proxy while others stored the result until they found another point. Clients outside the firewall either communicated via TCP/IP with a further proxy, also outside the firewall, or mailed the results to an e-mail address. Both the external and e-mail proxies periodically passed the results to the server. The client programs used little memory and a few bytes of disk space to log results. It did not interfere with users because the process was set to operate at very low priority. Thus any other task that the computer needed to do was given priority over the elliptic curve program. Clients were available for a range of operating system platforms, including Windows 95 and NT, MacOS, various Unixes and OpenVMS. Since TCP/IP sockets and e-mail were used as the mode of communication, the incompatibility issues were resolved.

Comprehensive, regularly updated statistics available from Web pages were an important factor in promoting interest in the problem and thus recruiting CPU resources. By presenting statistics on the number of distinguished

¹ Later developments in the cryptographic area include, in January 1999, the combined forces of Distributed Computing Technologies [2] and the Electronic Frontier Foundation [4] breaking a 56-bit DES algorithm in 22 hours, and, in February 1999, a research group breaking RSA-140, a 465-bit factor (see <http://www.rsa.com/pressbox/html/990119-1.html> and <http://www.rsa.com/rsalabs/html/rsa140.html> for more information).

points found by each machine, it was easy for participants to see their individual contribution. Each client could also choose to join a particular group. This encouraged groups to compete for the maximum contribution rate by recruiting other machines. One group even created its own statistics page. Roughly two out of every three machines that ever submitted a point kept running until the end.

Over 1200 machines from at least 16 countries took part in the ECCp-97 challenge. The answer was found in 53 days after 186 364 distinguished points had been submitted. This was about 45% of the expected number of distinguished points and roughly translates to 200 000 000 000 000 elliptic curve additions. The peak rate was 5000 000 000 000 which is equivalent to either 130 DEC Alphas 600 MHz 22164 or 460 Pentium II 300 MHz machines. Roughly 70% of the work was done by BT machines. The largest external teams were INRIA and Digital who both contributed approximately 8% of the total work.

The elliptic curve challenge clearly demonstrated the vast amount of idle computing power in a large company like BT. There were several areas where a better distributed architecture would have helped. During the elliptic curve challenge, the client software was continually being improved to ensure that it ran faster. The WindowsTM software had three different clients, each faster than the previous version. However, it was difficult to broadcast the availability of a new version and to persuade people to update. When the answer had been found the clients needed to be manually uninstalled (by running the uninstallation program). Some people were still running the client and sending results to the server for months afterwards. Recruiting resources relies on the installation of the program by the machine owner. The elliptic curve challenges obtained publicity and resources through being the first BT-wide distributive processing effort and by leading the world in company-wide distributive processing and cryptographic code breaking (ECCp-97 is still the largest elliptic curve cryptosystem to be broken). This would not be true for all distributive problems and it would certainly be harder to recruit someone's machine the second or third time you ask them to install a program.

4. Issues for a co-operative computing platform

The elliptic curve challenge showed that it is feasible to harness the spare cycles of machines at BTL. Different problems could be solved using a similar approach. What is needed is an architecture that can use a machine's spare cycles in a way that does not interfere with the owner of the machine. Therefore the distribution of the code, the communications needed to co-ordinate the various machines and the cleaning up of code once the problem has been solved should all be invisible to the owner.

The aim is to create a virtual supercomputer that can be used by groups who need massive processing power.

The following paragraphs detail the types of problem that can be categorised as solvable by parallel co-operative computing techniques, and the architecture needed. The typical network being considered is a large company's intranet, but this could be extended to co-operative computing over several co-operating internetworks, or the world-wide Internet.

The problem needs to be easily sub-divided into smaller sub-problems, independent or nearly independent of each other. This means that the only communication needed is the distribution of sub-problems and the collection of results. There are two reasons for this. Firstly, communication is almost always the slowest part of a parallel computing architecture, so problems using low communications provide the best results compared with a supercomputer. Secondly, as the network is used for normal activities such as e-mail and accessing the Internet, communications should be minimised to prevent network congestion. In the best-case scenario, clients will use little memory, disk space and communications overhead. If this is not the case, computers could work on the problem during non-work hours only and free up the resources for work hours. Distributed problems use a central co-ordinator to tell each machine what specific work to do.

There are three components in the distributed processing architecture. Clients are machines that contribute their spare cycles, working on a particular sub-problem. A distribution server controls the distribution of code to clients. A problem server is the central co-ordinator for an individual problem, telling each client what specific work to do, and collecting results. The interpretation of results could be carried out at the central co-ordinator, or be distributed to the clients if it is a difficult task. There are many common non-problem-specific messages between the three components. The problem server will need to tell the distribution server when to stop distributing code (when the problem has been solved), or when to change what is being distributed. Since there could be many problems being solved at any one time, there could be multiple distribution servers and many problem servers. Clients generally register to use one particular distribution server. A server could act as a proxy for another server at times when it would be otherwise inactive.

The elements to construct are a common communication framework, a code distribution architecture, a security framework, and tools for writing the problem code. For process-intensive problems, C/C++ or assembly code is required for maximum speed — tests have shown that a C implementation of basic cryptographic key-cracking algorithms is around ten times faster than the equivalent Java code even with Just-In-Time compilers. For other uses a language such as Java is appealing, both for its

cross-platform and security features. The solution is to offer both alternatives, and allow the client to be configured to what is acceptable. It is important that participants can keep informed on the progress of the problem, so that they can understand what they are contributing towards. A key part of the elliptic curve challenge was the statistics, which generated enthusiasm and interest. A central Web page could be created which contains the details of servers, current projects and links to information on these projects.

The main concern with a collaborative computing scheme is security — safeguards need to be in place to prevent malicious code from being downloaded. To that end, all code should be digitally signed (using public-key cryptographic methods). This ensures the integrity of code, prevents unauthorised parties from distributing code, and provides non-repudiation as deterrence. The distribution of keys can be securely achieved by using a ‘certification authority’. Important messages between machines can be signed and verified. Mobile agent architectures often use a language such as Java that can put restrictions on the actions of the downloaded code. However, for fast algorithms, code written in C or assembly language is necessary. It is important that users are able to choose the level of security with which they are satisfied. They can specify options such as allowing only code signed by a particular server, or only code that uses a security sandbox. Users can also choose to participate only at certain times, such as when the machine is not otherwise being used.

A different threat to malicious code is badly-designed code that unintentionally consumes system resources. A recent case in a telecommunications company in America of an employee using company computers to help find Mersenne Primes led to trouble when a memory leak started slowing the computers down. To this end it may be necessary for code to be checked and signed by an expert before it can be distributed. The regular deletion of all downloaded files (so that new files are downloaded) is an option the client could choose.

Communities who are worried about the security threat of downloading code from unknown servers could create their own server. This would act as a proxy for other servers, but source code would be required from the servers. This could then be checked and compiled before being used by the proxy. The distributed code would be signed by the proxy server to indicate that it has been checked and authorised, and is not malicious. The proxy server would act as a code ‘firewall’ for its clients. Communication with the main server would only be carried out by the proxy server, possibly on behalf of a particular client. The proxy server can be given many sub-problems to distribute to its clients. The idea of proxy servers is important, since it keeps the security problem within manageable bounds. It also allows groups to switch between their own and other problems with the minimum of effort.

5. Uses

There are several current uses for massive processing power other than cryptographic challenges. On the Internet there are groups working on finding Mersenne Primes [10] and helping in the search for extraterrestrial intelligence [11]. Commercial uses for co-operative computing are feasible with image rendering being a prime candidate. As each computer could work on a part of an image, it should fit into a co-operative computing architecture. Data mining is another possibility, if the data involved can be split into separate blocks. Due to the low communication requirement, the types of problem are more limited than with a traditional supercomputer, or even a group of purpose-built machines such as a Beowulf [12] cluster. However, there is no hardware cost involved, since the machines are necessary for individual use.

Such problems that need processing power as described above are not the only ones that could use this architecture. Software products that need to be scalable could be tested using distributed processing. A program to simulate clients could be distributed to test their reliability and scalability. If these dummy clients are created on just one computer, memory quickly runs out before it becomes a realistic simulation. Therefore using a little bit of memory from lots of computers is a better option. The clients can be co-ordinated by the problem server to simulate particular conditions at particular times. A further idea is use as a virtual world — each client computer holds part of the virtual world and movement between each part is handled by communication between the two computers holding each side of the boundary. As this may be a more communication-intensive problem, it may need special client permissions, or be used only at certain times.

6. Conclusions

This paper has considered the use of idle computing time to solve large-scale distributive problems. It has detailed several successful distributive attacks on cryptographic challenges both by Internet groups and a BT-led consortium. The ECCp-97 challenge and the successful attack on it has been extensively discussed, including lessons learned in the process. The extension of this approach to solve more general problems has been considered and a framework for co-operative computing proposed. The feasibility of linking computers in a secure manner to attack previously insoluble problems has been demonstrated and implementing this is a major possibility for the future.

References

- 1 Stinson D R: ‘Cryptography Theory and Practice’, CRC Press (1995).
- 2 Distributed Computing Technologies — <http://www.distributed.net>
- 3 RSA challenges — <http://www.rsa.com/rsalabs/html/challenges.html>

- 4 EFF DES Cracker — http://www.eff.org/pub/Privacy/Crypto_misc/DESCracker/
- 5 Certicom ECC challenges — <http://www.certicom.com/chal/index.htm> (1997).
- 6 BT's ECCp-97 challenge submission (March 1998) — <http://www.labs.bt.com/projects/security/crackers/p-97.txt>
- 7 Miller V: 'Uses of elliptic curves in cryptography', Advances in Cryptolog — CRYPTO '85, Lecture Notes in Computer Science, No 218, pp 417—426, Springer-Verlag (1986).
- 8 Koblitz N: 'Elliptic curve cryptosystems', Mathematics of Computation, No 48, pp 203—209 (1987).
- 9 van Oorschot P C and Wiener M J: 'Parallel collision search with cryptanalytic applications', Journal of Cryptography, to appear.
- 10 The Mersenne Primes organisation — <http://www.mersenne.org/prime.htm>
- 11 The Search for Extraterrestrial Intelligence (SETI) — <http://setiathome.ssl.berkeley.edu/>
- 12 Beowulf — <http://cesdis.gsfc.nasa.gov/linux/beowulf/beowulf.html>



Alexander Selkirk graduated from Balliol College, Oxford University, in 1994. He joined BT in 1996 having completed an MSc degree at Royal Holloway, University of London, in Information Security.

He wrote much of the software for the elliptic-curve key cracking challenge, including the results server and the client applications for Windows machines. He is currently working on the research and development of a secure collaborative computing scheme.



Adrian Escott graduated from Mansfield College, Oxford University in 1990. He then went to Royal Holloway, University of London where he completed his PhD in 1994. Before joining BT in early 1997, he lectured for two years at the University of Warwick.

During the elliptic curve key cracking challenge, he provided the cryptographic expertise and developed the core client code. He currently works on various cryptographic and security-related projects.